

Resumen global: Tarea 11 al 13

Tarea 11: Guía de Despliegue de HUE 4.11 en Ecosistemas Big Data

1. El Desafío Técnico: Incompatibilidad de Entornos

Tenemos una incompatibilidad entre entornos para ello tenemos que utilizar un contenedor Docker como un puente para ejecutar HUE de forma nativa sin degradar el sistema operativo principal.

2. Arquitectura del Laboratorio

La arquitectura se basa en una coexistencia de servicios entre el sistema anfitrión y el entorno aislado:

- **Kali Linux (Host):** Aloja el clúster de Hadoop.
- **Contenedor Docker (Ubuntu 20.04):** Aloja exclusivamente la interfaz de HUE.
- **Conectividad:** Se utiliza el parámetro `-network host` permitiendo que el contenedor vea y se conecte a los servicios de Hadoop

3. Hoja de Ruta del Despliegue (Roadmap)

Paso 1: Preparación del Host

Es obligatorio ejecutar estos comandos como usuario **ROOT**:

1. Actualización del sistema: `apt update`
2. Instalación de Docker: `apt install -y docker.io`
3. Activación del servicio: `systemctl enable --now docker`
4. Verificación de versión: `docker --version`

Paso 2: Creación del Entorno (La Máquina del Tiempo)

Se inicia el contenedor con acceso total a la red del host: `docker run -it --network host --name entorno-hue ubuntu:20.04 /bin/bash`

Paso 3: Descarga y Compilación de HUE

El proceso de construcción de HUE requiere precisión en las versiones de Python:

1. **Descarga:** Obtención del paquete `hue-4.11.0.tgz` vía `wget`.
2. **Extracción:** Descompresión mediante `tar xvf`.
3. **Compilación:** Ejecución de `make install` forzando la compatibilidad con `PYTHON_VER=python3.8`.

Paso 4: Corrección de Errores (Fixes)

Se ha identificado un error recurrente: `ModuleNotFoundError: No module named 'future'` para solucionarlo se instalan manualmente dependencias mediante pip: `pip install future daemon python-daemon pytz`.

Terminamos de generar archivos con el comando `make create-static`.

Paso 5: Configuración de Conectividad e Interoperabilidad

Conexión HUE a Hadoop (Dentro del Contenedor)

Se debe editar el archivo `/opt/hadoop/hue/desktop/conf/hue.ini` para apuntar a los nodos correspondientes:

- **HDFS:** `fs_defaultfs = hdfs://nodo1:9000`
- **WebHDFS:** `webhdfs_url = http://nodo1:50070/webhdfs/v1`
- **YARN:** `resourcemanager_host = nodo1`
- **HIVE:** `hive_server_host = nodo1` y `hive_conf_dir = /opt/hadoop/hive/conf`

Permisos de Proxy en Hadoop (En el Host Kali)

Es obligatorio editar el archivo `core-site.xml` en el host para permitir que HUE actúe como proxy:

- Configurar `hadoop.proxyuser.hue.hosts` y `hadoop.proxyuser.hue.groups` con el valor `*`.
- **Acción Crítica:** Se debe reiniciar el clúster Hadoop (`stop-all.sh` seguido de `start-all.sh`).

Paso 6: Arranque y Validación

Para iniciar HUE, se ejecuta el motor principal:

```
/opt/hadoop/hue/build/env/bin/supervisor -d
```

La validación se realiza accediendo al **browser** (navegador)

```
http://localhost:8888 .
```

Tarea 12: Análisis Técnico: El Puente Sqoop-Oracle en Ecosistemas Big Data

El Problema: OLTP vs. OLAP

El uso de Sqoop se justifica por la necesidad de separar las **cargas de trabajo** transaccionales de las analíticas:

Característica	OLTP (MySQL / Oracle)	OLAP (Hadoop / HDFS)
Uso Principal	Transacciones diarias / Ventas.	Análisis masivo de datos.
Tipo de Datos	Datos estructurados (Tablas).	Estructurados, semi y no estructurados.
Escalabilidad	Vertical (Hardware costoso).	Horizontal (Nodos económicos).
Riesgo	Consultas pesadas pueden bloquear el sistema.	Ejecuta algoritmos pesados (Spark, Hive) sin afectar producción.

Arquitectura y Preparación del Entorno

Fase 1: Cimientos y Variables de Entorno

Para que el sistema Linux reconozca los comandos de Sqoop y los binarios de Oracle, es obligatorio **configurar las variables de entorno** correctamente en archivos como `.bashrc` y `oracle_env.sh`:

- **Sqoop Home:** `export SQOOP_HOME=/opt/hadoop/sqoop`
- **Path:** `export PATH=$PATH:$SQOOP_HOME/bin`

El Traductor JDBC y Acceso

Sqoop requiere un intermediario para comunicarse con Oracle utilizando el driver JDBC `ojdbc6.jar`, el cual debe **copiarse** al directorio de librerías de Sqoop (`/opt/hadoop/sqoop/lib/`).

Es necesario desbloquear los usuarios de la base de datos mediante comandos SQL (`ALTER USER HR ACCOUNT UNLOCK`).

Operativa de Importación (Oracle a HDFS)

Anatomía del Comando Sqoop-Import

Un comando estándar se compone de los siguientes elementos:

1. **Acción:** `sqoop-import`
2. **Ruta (Conexión):** URL JDBC (ej. `jdbc:oracle:thin:@nodo1:1521:XE`).
3. **Llaves (Credenciales):** `-username` y `-password` .
4. **Objetivo:** La tabla a extraer (`-table DEPARTMENTS`).
5. **Destino:** Directorio en HDFS (`-target-dir /empleados`).

Optimización y Consultas Complejas

- **Control de Mappers:** En prácticas pequeñas, se recomienda usar `m 1` (un solo hilo) para evitar errores de claves primarias.
- **Filtrado de Datos:** Es posible realizar una importación selectiva usando `-columns` para filtrar columnas o `-where` para filtrar filas.
- **Free-form Query:** Permite extracciones mediante consultas SQL complejas (JOINS).
 - **Requisito Obligatorio:** Se debe incluir la cláusula `$CONDITIONS` para que Sqoop pueda inyectar sus propios límites y dividir el trabajo entre los mappers.
 - **Split-by:** Si se usa más de un mapper en una consulta libre, el parámetro `-split-by` es obligatorio.

Operativa de Exportación (HDFS a Oracle)

La exportación devuelve los datos procesados en Hadoop a la base de datos relacional para su uso en herramientas de Business Intelligence.

1. **Preparación en Oracle:** El administrador debe ejecutar primero un `CREATE TABLE` en Oracle con el esquema correspondiente.
2. **Ejecución:** Se utiliza el comando `sqoop export` especificando el directorio de origen en HDFS (`-export-dir`) y el delimitador de campos (`-input-fields-`).

`terminated-by ','`).

3. **Verificación:** Los resultados se confirman en Oracle mediante consultas estándar en `sqlplus`.

Hoja de Trucos de Sqoop (Referencia Rápida)

Tarea	Comando / Parámetro
Verificar versión	<code>sqoop version</code>
Explorar tablas	<code>sqoop list-tables --connect [jdbc-url] ...</code>
Importar tabla completa	<code>sqoop import --table [nombre]</code>
Filtrar columnas	Añadir <code>--columns "col1,col2"</code>
Filtrar filas	Añadir <code>--where "condicion"</code>
Consulta Compleja	Usar <code>--query "SELECT... WHERE \\${CONDITIONS}"</code>
Exportar a SQL	<code>sqoop export --table [nombre] --export-dir [ruta_hdfs]</code>

Tarea 13: Guía Técnica para el Despliegue y Gestión de Ecosistemas Big Data Multi-Nodo

1. Arquitectura del Clúster y Roles de Componentes

El ecosistema se estructura como una "pirámide de dependencias" donde el fallo de los niveles inferiores provoca el colapso de los superiores.

1.1 Estructura de Nodos

- **nodo1 (Maestro):** Centraliza la gestión de recursos y metadatos.
- **nodo2 (Esclavo 1):** Nodo de almacenamiento y procesamiento.
- **nodo3 (Esclavo 2):** Nodo de almacenamiento y procesamiento.

1.2 El Stack Tecnológico

Componente	Función Principal
ZooKeeper	Mantenimiento del Quórum y coordinación (Leader/Follower).
Hadoop (HDFS)	Almacenamiento distribuido.

Hadoop (YARN)	Gestión de recursos de computación.
HBase	Base de datos NoSQL sobre HDFS para búsquedas instantáneas.
Spark	Procesamiento distribuido rápido en RAM.

2. Protocolo de Despliegue Paso a Paso

Paso 0: Conectividad y Red

Antes de iniciar cualquier servicio, es obligatorio:

- **Configuración de Hosts:** El archivo `/etc/hosts` debe ser idéntico en los tres nodos, vinculando las IPs a sus respectivos nombres (`nodo1`, `nodo2`, `nodo3`).
- **Validación:** Ejecutar `ping -c 2 nodo2` y `ping -c 2 nodo3` para confirmar que las máquinas se reconocen por nombre.

Paso 1: Inicialización de ZooKeeper

ZooKeeper es el primer servicio en levantarse para coordinar el clúster .

1. **Asignar Identidad (myid):** Crear el archivo `/tmp/zookeeper/myid` con el ID correspondiente ('1' para maestro, '2' y '3' para esclavos).
2. **Arranque:** Ejecutar `zkServer.sh start` en todos los nodos.
3. **Quórum:** Se establece un *Leader* y dos *Followers* que se verifica con `zkServer.sh status` .

Paso 2: Hadoop (HDFS y YARN)

Se activan los servicios de almacenamiento y gestión de recursos mediante SSH desde el Maestro.

- **Comandos:** `start-dfs.sh` y `start-yarn.sh` .
- **Distribución de Procesos:**
 - **Maestro:** NameNode, SecondaryNameNode, ResourceManager.
 - **Esclavos:** DataNode, NodeManager.

Paso 3: HBase (La Base de Datos NoSQL)

HBase requiere una sincronización precisa con HDFS y ZooKeeper.

- **Funcionamiento:** El HMaster reside en el maestro, mientras que los HRegionServers se conectan a los DataNodes en los esclavos.

Paso 4: Explotación y Análisis (Spark)

- **HBase:** Los datos se insertan vía HMaster (`put`) y se almacenan físicamente distribuidos en los esclavos.
- **Spark sobre YARN:** Al ejecutar `pyspark --master yarn`, el Maestro no procesa; solicita memoria a los esclavos para realizar cálculos distribuidos.

3. Resolución de Problemas Críticos (Troubleshooting)

3.1 El Conflicto Java 21 vs. HBase

Java 21 introduce medidas de seguridad estrictas que bloquea HBase.

- **La Llave Maestra (Parche):**
 1. En `hbase-env.sh`: Exportar explícitamente `JAVA_HOME` y añadir `-add-opens java.base/java.lang.reflect=ALL-UNNAMED` a `HBASE_OPTS`.
 2. En `hbase-site.xml`: Configurar `hbase.wal.provider` como `filesystem`.
 3. **Distribución:** Es vital usar `scp` para copiar estos archivos a los esclavos antes del arranque.

3.2 Matriz de Diagnóstico Rápido

Síntoma (Log)	Diagnóstico	Solución Inmediata
"No route to host"	Red caída o cambio de IPs dinámicas.	Verificar IPs con <code>ip a</code> y actualizar <code>/etc/hosts</code> .
"ServerNotRunningYetException"	Falta de paciencia o caché de ZooKeeper corrupta.	Esperar 60s. Si persiste, limpiar en <code>zkCli.sh</code> (<code>deleteall /hbase</code>) y reiniciar.
Consola colgada en comando <code>put</code>	HRegionServer caídos por seguridad de Java 21.	Aplicar el parche en <code>hbase-site.xml</code> y reiniciar.
Error: "myid file is missing"	Síndrome del Director Amnésico.	El directorio <code>/tmp</code> se vació. Recrear carpeta y archivo <code>myid</code> .

4. Matriz de Validación y Reglas de Oro

4.1 El Checklist JPS Definitivo

Para confirmar que la arquitectura está "perfecta", el comando `jps` debe mostrar los siguientes demonios activos:

Capa / Servicio	Maestro (nodo1)	Esclavos (nodo2, nodo3)
ZooKeeper	QuorumPeerMain	QuorumPeerMain
HDFS	NameNode, SecondaryNameNode	DataNode
YARN	ResourceManager	NodeManager
HBase	HMaster	HRegionServer

4.2 Las 4 Reglas de Oro para la Operatividad

1. **El Orden es Inquebrantable:** La secuencia obligatoria de arranque es: Red - > ZooKeeper → Hadoop → HBase → Spark. Nunca se debe alterar.
2. **'jps' es tu Sexto Sentido:** No avanzar al siguiente paso sin confirmar visualmente que los demonios están vivos en todas las máquinas.
3. **La Regla del Minuto:** Dar tiempo a los sistemas distribuidos (especialmente HBase) para sincronizarse. La impaciencia genera "errores fantasma".
4. **Limpiar antes de Reiniciar:** Si algo falla, eliminar procesos perdidos con `pkill -9` y limpiar la carpeta `/tmp` antes de reintentar el arranque.